

— TECH TALK · 60 MIN

# OpenClaw 编排 深度实践

轻量化容器化 Agent · 配置驱动的多智能体路由 · 自循环测试链路 · Copilot 协同

**Kinfey Lo**

Microsoft Senior Cloud Advocate

2026

# 本场议程 · Five Acts in 60 Minutes

01

## 解构 OpenClaw

轻量化、容器化的 Agent 运行机制与三层架构

02

## 状态机与拓扑设计

不靠大型框架，仅用配置文件实现多 Agent 路由

03

## 自循环测试链路

Coder → Tester → Executor 三智能体闭环

04

## Copilot Snippet 驱动

快速生成 agent.yaml 与 tool\_definition.json

05

## CoT Prompt 注入

通过 Copilot 生成复杂思维链模板

# OpenClaw 是什么？

## 一句话定义

一个跨通道的 AI Agent 网关运行时：

用 **JSON5 配置文件** 定义 Agent、模型、工具、通道、Skills 与沙箱；

所有运行单元都跑在  
**本地容器** 中，按最小权限加载工具。

开箱支持 WhatsApp / Telegram / Discord / Slack 等通道接入。

## Local-First

默认本地 Gateway，端口 18789，loopback 绑定

## Config-Driven

openclaw.json 即真理之源，热重载支持

## Sandbox by Default

Docker 镜像 openclaw-sandbox · 只读根 · network: none

## Multi-Runtime

pi (内置) · codex · claude-cli 三种 Harness

## Skill System

可插拔技能：github、weather、peekaboo ...

# 轻量化 + 容器化：默认就开启的最小权限

OpenClaw 的 sandbox 不是可选项 — 它是 agent 的默认外壳

|                        |                               |
|------------------------|-------------------------------|
| <b>scope: session</b>  | 每个会话独立容器，会话结束销毁               |
| <b>readOnlyRoot</b>    | 根文件系统只读，写入仅限 tmpfs            |
| <b>network: none</b>   | 默认无网络，工具显式启用才放行               |
| <b>user: 1000:1000</b> | 非 root 运行，特权被剥离               |
| <b>tmpfs</b>           | /tmp · /var/tmp · /run 内存级临时盘 |

```
// openclaw.json (excerpt)
agents.defaults.sandbox = {
  mode: "non-main",
  scope: "session",
  workspaceRoot:
    "~/.openclaw/sandboxes",
  docker: {
    image:
      "openclaw-sandbox:bookworm",
    workdir: "/workspace",
    readOnlyRoot: true,
    tmpfs: ["/tmp", "/run"],
    network: "none",
    user: "1000:1000",
  },
  browser: { enabled: false },
}
```

# Provider / Model / Runtime / Channel — 四层职责清晰

## Channel

消息进出 OpenClaw 的入口

Telegram · Slack · Discord · WhatsApp

## Agent Runtime

驱动模型循环、处理原生 Tool Calls

pi (内置) · codex · claude-cli

## Model

本次 Agent Turn 选用的模型

claude-opus-4-6 · gpt-5.4 · llama-3.1

## Provider

鉴权、模型发现、模型引用命名

anthropic · openai · openai-codex

*Runtime ≠ Provider — Runtime 是底层执行循环；Provider 是鉴权与目录服务。*

# openclaw.json — 一个文件描述整个 Agent 系统

```
{
  identity: { name: "Clawd", emoji: "🔥" },
  agents: {
    defaults: {
      workspace: "~/.openclaw/workspace",
      model: { primary: "anthropic/claude-opus-4-6" },
      skills: ["github", "weather"],
      sandbox: { mode: "non-main", ... },
    },
    list: [
      { id: "main", default: true,
        thinkingDefault: "high" },
      { id: "quick", fastModeDefault: true,
        skills: [] },
    ]
  },
  tools: { allow: ["exec", "read", "write", ...],
            deny: ["browser", "canvas"] }
}
```

## identity

品牌人格 · 决定回复语气

## agents.defaults

所有 Agent 的基线配置（继承）

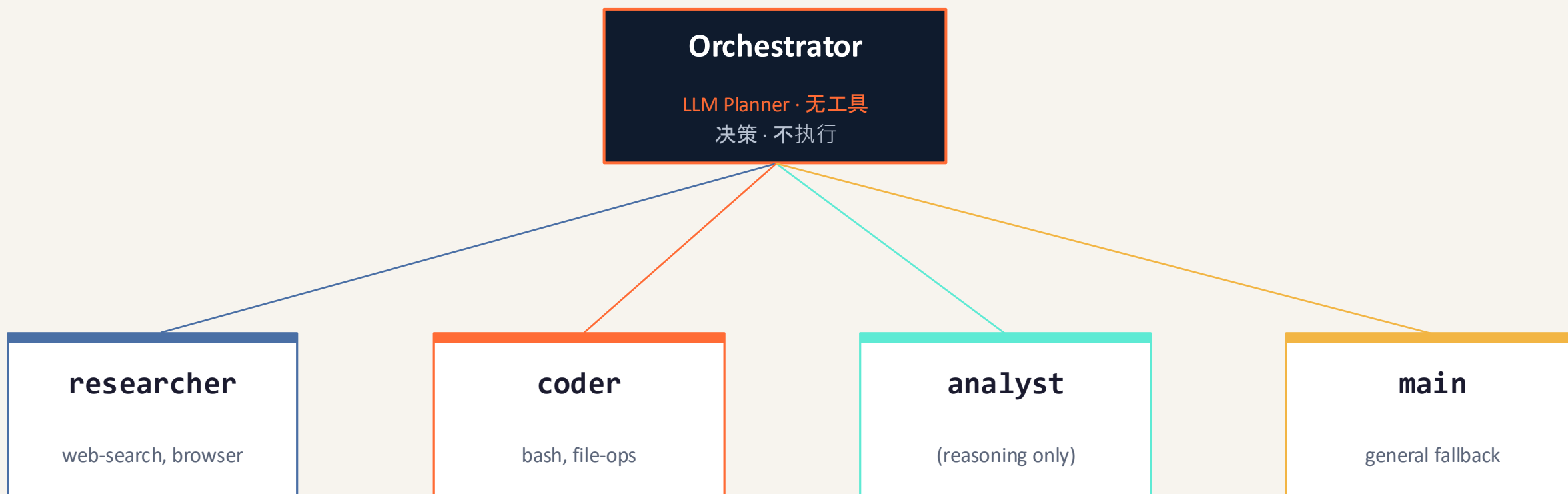
## agents.list[]

具名 Agent · 可单独覆盖默认值

## tools

全局白/黑名单 · 工具可见性总开关

# 多 Agent 路由 — 由 Orchestrator 居中调度



Agents 互不感知 · 仅接收任务并返回结果。Orchestrator 通过 name 或 capability 路由。

# Think → Execute → Result · 自适应循环



```
// LLM Planner 输出格式
{ "action": "execute",
  "tasks": [ { "agent": "coder", "task": "实现 URL shortener" },
              { "agent": "researcher", "task": "对比 Redis 与 SQLite" } ] }
{ "action": "finish", "answer": "..." }
```

# 为什么不用 LangGraph / AutoGen ?

## Heavy Framework

- ✗ 需要 Python 环境 + 多个依赖
- ✗ 拓扑硬编码在代码里
- ✗ 改路由 = 改源码 + 重启
- ✗ 调试需要进 IDE 单步
- ✗ 每个新工具都要写 Tool 类

## OpenClaw Way

- ✓ 单二进制 + 一个 JSON5 文件
- ✓ 拓扑写在 `agents.list[]`
- ✓ 改路由 = 编辑 JSON + 热重载
- ✓ Dashboard 上看 SSE 流
- ✓ 工具走 allow/deny 名单声明

# agent.yaml — 一个 Agent 的全部基因

```
id: coder
default: false
workspace: ~/.openclaw/workspace-coder
model:
  primary: anthropic/claude-opus-4-6
  fallbacks: [openai/gpt-5.4]
skills: [github, shell-utils]
thinkingDefault: high
reasoningDefault: on
fastModeDefault: false
elevatedDefault: on
timeoutSeconds: 600
maxConcurrent: 3
soul: |
  你是资深 Coder, 遵循 SOLID,
  写出可被 Tester 直接调用的纯函数。
tools: { allow: [exec, read, write, edit] }
```

## id / default

唯一标识 · default Agent 接管未路由请求

## model

primary + fallbacks · 模型不可用时自动降级

## skills

可插拔技能集 · 决定能调用哪些外部能力

## thinking / reasoning

思考链可见性与深度 (off / low / high)

## soul

System Prompt 来源 · 角色定义的灵魂

## tools.allow

Per-Agent 工具白名单 · 覆盖全局

# tool\_definition.json — 最小权限的契约

```
{
  "name": "run_pytest",
  "description":
    "在沙箱内执行 pytest 并返回结构化结果",
  "inputSchema": {
    "type": "object",
    "properties": {
      "path": { "type": "string" },
      "marker": { "type": "string" }
    },
    "required": ["path"]
  },
  "runtime": {
    "command": "pytest",
    "args": ["{{path}}", "-m", "{{marker}}"],
    "timeoutSec": 600,
    "sandbox": true
  }
}
```

## 三条铁律

### 01 Schema First

工具的输入参数必须是 JSON Schema · LLM 才能正确填槽

### 02 Sandbox by Default

runtime.sandbox: true · 不显式开沙箱 = 默认拒绝

### 03 Explicit Timeout

timeoutSec 必填 · 防止失控工具吃完资源

# Principle of Least Privilege — 应用到每一个 Agent

Gateway 层

`auth.token + Tailscale + loopback` 绑定

Agent 层

`tools.allow / deny + skills` 白名单

Tool 运行层

`Docker sandbox + readOnlyRoot + network: none`

一次 *prompt injection* · 最多吃掉一个会话级容器，不污染主机。

# Coder → Tester → Executor — 三智能体闭环



Orchestrator 仅维护任务队列与终止条件 — 三个 Agent 互不调用，仅交换文件与结果

# Coder — 把意图变成可读、可测的代码

## SOUL.md

你是高级 Python 工程师。

约束：

- 一次只写一个函数
- 必须有完整 `type hints` 与 `docstring`
- 不写 `try/except` 兜底，让错误暴露
- 输出必须是纯文件（无 `main` 块）

输出格式：

- 1) 文件名：`src/xxx.py`
- 2) 完整源代码
- 3) 一句话说明该函数的输入/输出契约

拒绝：解释、教程、与代码无关的对话。

## Tools

write

edit

read

## 失败信号

### syntax\_error

import 解析失败 → 重写文件

### test\_failed

Executor 反馈 trace → 修补该函数

### type\_mismatch

Tester 检测到契约不符 → 调整签名

# Tester — 只写测试，不允许碰实现

## 职责契约

读取 `src/*.py` 中的函数签名 → 生成 `tests/test_*.py` · 不允许修改 `src/`

## CoT Prompt 模板

```
Step 1: 阅读 {{file}}, 列出所有公开函数
Step 2: 对每个函数, 列举
    · 正常路径输入
    · 边界值 (空、None、极大、极小)
    · 非法输入 (错误类型)
Step 3: 用 pytest.parametrize 生成测试
Step 4: 输出文件 tests/test_<module>.py
    · 不要包含 fixture 之外的代码
    · 每个测试都要 assert, 不许 print
```

## 生成示例

```
import pytest
from src.calc import safe_div

@pytest.mark.parametrize("a,b,exp", [
    (10, 2, 5),
    (-9, 3, -3),
    (0, 5, 0),
])
def test_safe_div_ok(a,b,exp):
    assert safe_div(a,b) == exp

def test_safe_div_zero():
    with pytest.raises(ValueError):
        safe_div(1, 0)
```

# Executor — 在 OpenClaw 沙箱中执行并反馈

01

## 拉起沙箱

openclaw-sandbox:bookworm-slim · scope: session

02

## 挂载产物

src/ + tests/ 进入 /workspace 只读复制

03

## 执行 pytest

pytest -q --json-report --tb=short --timeout=60

04

## 结构化回灌

把 status / failed / trace 喂给 Orchestrator

```
{
  "status": "failed",
  "summary": {
    "total": 8,
    "passed": 6,
    "failed": 2
  },
  "failures": [
    {
      "test":
        "test_safe_div_zero",
      "trace":
        "DID NOT RAISE ...",
      "hint_for_coder":
        "修补 b==0 路径"
    }
  ]
}
```

# 用注释驱动 Copilot 直出 agent.yaml

写下注释 ↓

```
# OpenClaw agent definition
# role: tester
# allowed tools: write, read
# model: claude-opus-4-6
# cot: parametrize edge cases
# ---
```

Copilot 直接补全 ↓

```
id: tester
default: false
model:
  primary: anthropic/claude-opus-4-6
tools: { allow: [write, read] }
thinkingDefault: high
```

提示词工程的三个杠杆

## 上下文锚点

首行写明「OpenClaw agent definition」让 Copilot 锁定 schema

## 字段提示

用注释列出 role / tools / model 三件套，缺一不可

## 分隔符 ---

结尾 --- 触发 Copilot 进入「补全 YAML 文档」模式

## 版本注释

加一行 # schema: openclaw v1 锁定字段命名

# tool\_definition.json — 让 Copilot 写 JSON Schema

1

## 选定函数

在 IDE 中高亮 Python/TS 函数签名

2

## 触发提示

`/* generate openclaw tool def */` 注释

3

## 校对 schema

Copilot 自动从类型注解推断 JSON Schema

4

## 加沙箱字段

手动补 `timeoutSec`、`sandbox: true` 两项

源函数 → 生成 schema

```
def run_pytest(
    path: str,
    marker: str | None = None,
    timeout: int = 60,
) -> PytestResult:
    '''run pytest in sandbox'''
    ...
```

```
{ "name": "run_pytest",
  "inputSchema": {
    "type": "object",
    "properties": {
      "path": { "type": "string" },
      "marker": { "type": ["string", "null"] },
      "timeout": { "type": "integer" },
      "required": ["path"] } } }
```

# 用 Copilot 生成可复用的思维链模板

## 给 Copilot 的 Meta-Prompt

```
/*
 * 帮我生成 Chain-of-Thought 模板
 * 角色: {{ role }}
 * 输入: {{ input_shape }}
 * 输出: {{ output_shape }}
 * 必须包含步骤:
 *   1) Restate the goal
 *   2) List unknowns / assumptions
 *   3) Decompose into  $\leq 3$  sub-tasks
 *   4) Solve each (with verification)
 *   5) Synthesize · Self-check
 * 禁止: 直接给答案、跳步、隐式假设
 */
```

## 生成的 Prompt 模板

你是 {{role}}。请按以下步骤思考：

1. 用一句话复述目标。
2. 列出已知 / 未知 / 假设。
3. 拆成不超过 3 个子任务。
4. 逐个解决：
  - 给出方案
  - 给出验证方法
  - 标注置信度 (0-100)
5. 综合结论 · 自检矛盾点。

若任意步骤置信度 < 75 →  
调用 ask\_user 工具求证，禁止猜测。

# 三个带回家的钩子

01

配置文件就是拓扑

改路由不要写代码，改 `agents.list[]`

02

Sandbox 不是奢侈品

默认开启就是默认安全，最小爆炸半径

03

Copilot 是 schema 引擎

用注释锚点 + 类型签名让它写出可信 YAML/JSON

## Q & A

[docs.openclaw.ai](https://docs.openclaw.ai) · [github.com/zeynepyorulmaz/openclaw-orchestrator](https://github.com/zeynepyorulmaz/openclaw-orchestrator)

Kinfey Lo · [kinfeylo@microsoft.com](mailto:kinfeylo@microsoft.com)